

execlab

A historical execution-benchmarking laboratory — project context and complete feature inventory for a new contributor

1. What execlab is

execlab is a historical execution-benchmarking laboratory. A user brings the output of a trading strategy — a CSV of timestamped parent-order instructions — picks a historical session and an execution algorithm, and the platform executes those instructions minute by minute against real 1-minute market data. It then reports how much of the strategy's theoretical profit actually survived the cost of trading.

The distinction that organises the whole codebase: the user's alpha model decides WHAT to trade and is never seen by us; execlab measures HOW WELL that decision was carried out. Two people can run the identical instructions on the identical day and get materially different results purely from execution choices, and the platform's job is to make that difference visible and explain it.

It is written in OCaml in the Jane Street style (Core, ppx_jane, dune, expect tests). The browser client is Bonsai compiled with js_of_ocaml, so the entire simulator runs client-side; the same libraries also compile natively for a CLI and a server. Roughly 10,600 lines across the libraries, binaries and client.

2. Vocabulary a reader needs first

Parent order	One instruction from the alpha file: buy 5,000 TSLA between 10:00 and 11:00. It activates at its arrival time and must finish by its deadline.
Child order	A slice the algorithm sends to the market. Parents are worked by many children.
Arrival price	The market price at the moment the parent activated — the benchmark everything is judged against, because it is the price the decision was made at.
Implementation shortfall	Average fill price versus arrival price, side-adjusted so positive always means worse. The headline cost number.
Taker / Maker	Whether a fill crossed the spread (paying it) or rested and was hit (earning it). Recorded per fill because it cannot be reconstructed later.
Value-add	Net P&L minus what the naive 'immediate' baseline would have made on the identical instructions. This is the score.

3. Architecture at a glance

Ten libraries under lib/, each with src/ and test/ subdirectories, plus four executables and one br
Dependencies flow strictly upward; nothing below knows about anything above it.

lib/types	The shared vocabulary: Price (integer cents), Size, Side, Symbol, Order_id, Fill, Order_type, Time_in_force, Alpha_instruction. Private types with validating constructors.
lib/market	Market_bar, Trading_day (exactly 390 strictly-ascending minutes, 09:30–15:59), Data_loader, Day_stats (VWAP, volume profile, realized volatility).
lib/alpha	Parser for the user's instruction CSV. Tolerant of shape (optional header, blank lines, padding), strict about content, and reports every bad row with its line number.
lib/execution	Child_order, Parent_order (a state machine), Order_manager, Algorithm_intf, and the five algorithms.
lib/simulation	Engine A (Fill_model), the Engine_intf seam, and Driver — the per-bar loop that drives a whole session.
lib/exchange	Engine B: Book, a deterministic 32-bit RNG, and Synthetic_market — a real order book with market-making bots.
lib/analytics	Portfolio, Benchmarks, Transaction_cost (the full cost tree), Report.
lib/session	The single run pipeline — algorithm construction, execution, grading — shared by the CLI, the browser and the server so the three fronts cannot drift apart.
lib/protocol	The one client/server seam: sexp request and response types for six endpoints.
lib/server	Catalog (market data on disk), Store (run persistence), Accounts, a minimal HTTP server, and the request router.

4. The five execution algorithms

All implement one interface. Each bar they receive the current time, the PREVIOUS bar (never the current one — that would be look-ahead), the parent's state, and its live children; they return actions. All five share the same catch-up machinery: compute a target position, subtract what is already filled or working, and the deficit as a market IOC child so that anything the market refuses this minute is re-demanded next minute.

TWAP	Time-weighted. Target is total x elapsed / window, in integer arithmetic. Assumes nothing about the market; the reference against which the others are judged.
VWAP	Volume-weighted. The same schedule with the clock replaced by a forecast volume curve, so it trades more when the market is typically busy. The forecast is the leave-one-out average of the symbol's OTHER sessions — never the day being traded — so it can genuinely be wrong.
POV	Percentage of volume. No schedule at all: it demands a fixed share of the volume that has actually traded since arrival, so it adapts to real liquidity but promises no finish time. Floors rather than rounds, because the rate is a ceiling.

Implementation shortfall	Front-loaded. The Almgren-Chriss optimal trajectory: scheduled remaining quantity decays as $\sinh(u(1-f))/\sinh(u)$ across the window, with all the theory's parameters collapsed into one dimensionless 'urgency' knob. Urgency 0 is exactly TWAP (it runs TWAP's own arithmetic); large urgency approaches Immediate.
---------------------------------	--

Immediate	The naive baseline: send the entire remaining quantity the moment the parent activates. Every run is automatically graded against it, and it is a real competitor, not a strawman — it wins whenever the signal decays faster than a schedule can work.
------------------	---

5. Two fill engines behind one interface

The driver asks a market only two questions — 'a new minute is starting, do any resting orders is a new order, what does it fill at?' — so engines are interchangeable and no algorithm can tell it is trading in.

Engine A — the bar model (deterministic calculator)

- **Pricing** A marketable order fills at the bar's open, plus half the fabricated spread, plus a square-root impact penalty: $25c \times \sqrt{\text{order size} / \text{bar volume}}$.
- **Participation cap** No more than 10% of a bar's volume may be consumed, shared across all orders in that minute. The remainder of an IOC child is cancelled and re-demanded next bar.
- **Resting limits** Fill only if the bar trades strictly through the limit price, tagged Maker.
- **Knobs** `half_spread` (2c), `max_participation` (0.10), `impact_coefficient` (25c) — all user-editable in the UI.

Engine B — the synthetic exchange (emergent)

- **The book** Three market-maker archetypes ladder real price levels around each bar's price path, showing roughly 12% of the bar's volume per side. Depth scales with the bar's volume; rung width is $0.42 \times \sqrt{\text{bar range}}$, which puts the catalog's median minute on Engine A's 2c half-spread.
- **Impact emerges** A client order walks the ladder level by level. It pays more because it exhausted the cheap sellers, not because a formula said so. Run out of book and you simply get less.
- **Noise traders** Six prints per bar, sized from the bar's volume with a directional bias, generate the tape.
- **Temporary impact** Makers requote around the historical path each bar, so a dent heals.
- **Permanent impact** Taking liquidity accumulates signed participation ('pressure'), which shifts the quote centre by $\text{coefficient} \times \sqrt{\text{pressure}}$ and decays 40% a bar — information leaking into the price, so a buyer who keeps lifting finds the offers rising.
- **Queue position** A resting client limit records the agent size displayed at its price when it arrived and fills only from flow that reaches it once that queue is served. Flow is finite and awarded by price priority. A limit priced through the touch crosses instead of resting.
- **Determinism** All randomness comes from a pure 32-bit LCG, so the same seed yields bit-identical fills natively and in the browser — which the server's re-run-to-verify anti-cheat depends on.
- **Calibration invariant** With zero client orders, the simulated tape's VWAP tracks the historical session's VWAP within 5bps. This is pinned by tests, including on a wide-range day.

6. What the platform measures

Every run produces a full cost tree per parent order in exact integer cents, satisfying two identities that are enforced by tests:

$$\text{gross alpha} = \text{net P\&L} + \text{shortfall} + \text{opportunity cost}$$
$$\text{shortfall} = \text{timing} + \text{spread} + \text{impact}$$

Gross alpha	What the idea deserved: the full quantity filled instantly and free at the arrival price, valued at the session close.
Net P&L	What actually happened: filled shares valued at the close, minus what they really cost.
Shortfall / friction	What execution cost on the shares that did fill, in cents and in side-adjusted basis points.
Timing	The drift half: what the market's own movement between arrival and each fill cost. Negative when the tape moved your way.
Spread	The half-spread toll paid on liquidity-taking fills. A synthetic run attributes zero here — the exchange has no configured toll — so the whole residual lands in impact.
Impact	What your own trading pushed the price, beyond drift and spread.
Opportunity cost	What the unfilled remainder never earned. Pure lost alpha when the idea was right; a lucky escape when it was wrong.
Completion rate	Filled divided by requested.
VWAP slippage	Average fill versus the session's own VWAP: did you trade worse than the crowd?
Alpha capture	Net divided by gross. Blank when gross is not positive, since the ratio is then meaningless.
Value-add	Net P&L minus the immediate baseline's net P&L on the same instructions. The competitive score.

7. The browser client, screen by screen

A single-page Bonsai app in `app/ui/`. The whole simulation runs in the browser: the market data cache embedded into the bundle at build time, so a run needs no server at all. Eight screens, with a light theme toggle present on every one of them.

Landing

- **Purpose** The front door: what the lab is, how a run works, the five algorithms, the two fill engines, and a strip of headline statistics.
- **Accounts** Create an account or sign in, inline. Guests may use the entire laboratory; an account is required only to publish a run to a leaderboard.

- **Entry** Buttons into the wizard or, when signed in, the dashboard.

Dashboard

- **Recent runs** Every completed run this browser has done, newest first, with symbol, day, algorithm, value-add, net P&L and capture. Persisted in localStorage, so it survives reloads.
- **Entry point** Starts a new simulation.

Choose day

- **Symbol browser** All six bundled symbols; picking one shows its sessions.
- **Calendar** A real month calendar with month navigation, where only sessions that exist in the catalog are selectable. Hover pops the cell.
- **Why day first** The day is chosen before the alpha so every later step can validate against it.

Alpha instructions

- **Three ways in** Paste the CSV directly, upload a file from disk, or start from one of six built-in samples ('A typical day', 'A morning of buying', 'Buy now, sell later', 'Selling into the close', 'Busy day, six orders', 'Three at once'). Samples are generated for the chosen symbol so they can never trip the symbol check.
- **Live preview** The parsed instruction table appears as you type; a malformed file shows the parser's line-numbered errors instead.
- **Format** arrival_time,symbol,side,quantity,deadline — with an optional header row, blank lines and padding all tolerated.

Setup

- **Algorithm** Pick one of TWAP, VWAP, POV, IS, Immediate.
- **Fill engine** Bar model or synthetic exchange, with a seed field for the latter.
- **Presets** Three plain-language stances — Passive, Balanced, Aggressive — that set the strategy's own dials. They deliberately never touch the fill-model numbers, because those are the market, not the trader.
- **Parameters** Editable fields for the fill model's half-spread, participation cap and impact coefficient, plus whichever knob the chosen algorithm uses (POV rate or IS urgency). Validated on Run; a bad value surfaces as an error, never a crash.
- **Instruction summary** The parsed alpha shown again for confirmation before committing.

Simulation replay

The centrepiece. The run executes instantly and deterministically; this screen replays the result minute by minute on a clock.

- **Transport** Play/pause, speeds of 1x, 4x and 16x, a scrubber bound to the session minute, a live clock, and a status line (replaying / paused / session complete).
- **Chart zoom** Full day, or a 2h / 1h / 30m / 15m window that follows the playhead.
- **Price chart** The session's price line drawn up to the playhead, with a dashed day-VWAP reference, dollar gridlines and hourly time labels.
- **Order windows** Each parent's arrival-to-deadline span shaded in its own identity colour, with its arrival price annotated. Past three orders, or whenever windows overlap, the display switches to compact stacked lanes so it cannot turn to mud.

- **Crosshair** Hovering anywhere gives a dashed vertical, a dot on the price line, and a panel showing that minute's close, open, high, low and volume — plus a coloured line for every order window covering that moment, whose shading brightens.
- **Fill overlay** A toggle that draws every fill as a dot at its executed price, coloured by owning order.
- **Live readouts** Total mark-to-market P&L updating as the replay runs; execution benefit versus the immediate baseline is withheld until the close so it cannot spoil the ending.
- **Orders table** One row per parent: side and quantity, window, arrival price, average fill, filled with percentage, slices, shortfall and VWAP slippage in bps, status, and a progress bar in the order's colour.
- **Event log** The session narrated in plain words — arrivals, 'bought 83 @ \$395.71 (taker)', completions with the final average, expiries — newest first, scrollable, tagged by order.

Results

- **Verdict** One plain-language sentence saying what happened, followed by four large tiles: gross alpha, net P&L, capture and value-add.
- **Execution cost breakdown** One row per order showing the arithmetic left to right: timing + spread + impact = shortfall, with the bps figure alongside and a totals row.
- **Results table** The P&L identity per order: gross alpha, shortfall, opportunity, net P&L, value-add versus immediate, and capture.
- **Metrics glossary** Thirteen entries, each giving a plain-English meaning and then the precise definition — openable from the results screen for readers who do not speak TCA.
- **Leaderboard** The board for this exact day, alpha, engine and simulator calibration, ranked by value-add. Signed-in users can publish; guests are told why they cannot.
- **CSV export** Two downloads, generated entirely in the browser. results.csv carries one row per order with identity, window, benchmarks, average fill, bps figures and the entire cost tree in exact integer cents; fills.csv is the raw blotter, every fill tagged with its owning order.
- **Navigation** Back to the replay, on to a new simulation, or to the dashboard.

My runs

- **Notebook** A signed-in user's saved runs, kept server-side, with which are published.
- **Reopen** Any saved run can be opened again.

8. The server and the leaderboard

bin/server.exe is a small blocking HTTP server that serves the client application AND the API from one process, replacing the need for a separate static file server. Six sexp endpoints:

POST /api/create-account	Register a username and passcode; returns a session token.
POST /api/sign-in	Reissue a session for an existing account. The error deliberately does not distinguish a wrong name from a wrong passcode.
POST /api/save-run	Store a run in the caller's private notebook.
POST /api/my-runs	List that notebook.

POST /api/submit-run	Publish to a leaderboard; returns the server's own grading and the refreshed board.
POST /api/leaderboard	Read a board.

The design decision worth understanding: the server never trusts a client's numbers. A server only the run's CONFIGURATION, and the server re-executes it. Because the simulator is the config is the proof of its own score. It also grades under house physics — the fill model price and a synthetic run's seed derived from the board's identity so nobody can shop for a price while leaving the algorithm's own knobs free, since those are the strategy rather than the price.

Boards are directories, keyed by symbol, date, alpha hash, engine and a digest of the price; the simulator therefore starts a fresh contest instead of ranking runs from two different markets. Persistence is sexp files; corrupt or foreign files are skipped rather than fatal. Boards hold the whole record so a submission can be audited, but what is published is only who, when and how a strategy stays with its owner.

9. Command-line tools

bin/main.exe	Parse an alpha file, or run a full session and print the comparison report against the immediate baseline. Takes a symbol, date, algorithm and engine.
bin/server.exe	The client and API on one port.
bin/sweep.exe	Runs every scenario across every session, algorithm and engine — 5,940 runs — asserting the invariants that must hold of any execution, and printing a league table. Exits non-zero on any violation, so it is a regression net over the real data that the sandboxed expect tests cannot reach. <code>http://localhost:8081/</code>
<code>dune exec bin/main.exe - examples/demo_alpha_tsla.csv -SLA 2026-07-09 -Is synthetic.7</code>	
<code>dune exec bin/server.exe 8081</code>	
<code>dune exec bin/sweep.exe</code>	<code># invariant sweep + league table</code>
bin/demo.exe	Runs the seven-scenario demo library and prints the comparison tables.
<code>dune exec bin/demo.exe 1</code>	<code># on demo scenario</code>
<code>dune build app/ui/main.bc.js</code>	<code># rebuild the browser bundle</code>

10. The demo library	
demos/ holds seven scenarios, each a real session chosen by measuring the catalog, each isolating one variable, and each documented in demos/README.md with its market conditions, the comparison, the measured outcome, the metric that reveals it, and the assumptions that make it fair. Several end with the obvious answer losing, which is the point.	
1 VWAP on a U-shaped day	META 2026-07-10, where 40% of volume trades in the first and last half hour. VWAP tracks the benchmark at 3.2bps against TWAP's 7.4.
2 A flat day	META 2026-07-17, the flattest profile in the catalog. VWAP's forecast misfires and TWAP tracks better — case 1's honest counterweight.
3 POV and a volume spike	One minute trading sixteen times the median. POV books the lowest impact of the three and the worst shortfall: cheapest footprint, worst drift.

4 Adverse drift	NFLX 2026-07-17, the tape running 256bps against the buyer in an hour. Shortfall falls monotonically with IS urgency, and the timing column explains why.
5 A fast-decaying signal	META 2026-07-09, where 125 of the hour's 153bps land in five minutes. Immediate wins outright at 1.7bps.
6 Three deadlines	One order, one algorithm, three deadlines. Impact per share rises 4.75c to 8.00c and completion falls to 39.7% as the deadline tightens.
7 Participation rate	POV at 2%, 6% and 15%: completion 22.5% to 100% while impact per share rises 4.34c to 8.00c. The trade-off with nothing else changed.

11. Market data

data/ holds six symbols (AAPL, GOOG, META, MSFT, NFLX, TSLA) of roughly eleven sessions, about 82,000 one-minute bars — as canonical per-day CSVs with a header of time,open,high,low,close,volume. Trading_day enforces exactly 390 strictly ascending minutes, 15:59, so a malformed file cannot enter the system. The browser bundle embeds the entire time.

12. What the simulator does and does not claim

The honest product description is 'execution strategies run in a synthetic market calibrated to a session'. We do not reconstruct the real order book. The documented approximations are: a spread, an unknown intra-bar price path, optimistic passive fills in Engine A, the participation for liquidity, and stateless impact in Engine A. These distort absolute costs identically for everyone, so rankings and value-add remain meaningful even though 'your true P&L' is not. That is the fair comparison.

Deliberate guarantees a contributor should not break: algorithms see only the previous bar, excludes the day being traded; the arrival benchmark is sampled at the first minute in which they actually begin; and both cost identities hold exactly in integer cents.

13. Conventions for anyone writing code here

- **Style** CLAUDE.md is authoritative. Core as the standard library, open! Core everywhere, snake_case, type t per module, mli for every module with real documentation, no polymorphic compare, Or_error for fallible boundaries and _exn only where a failure is a programming bug.
- **Testing** Test-first for anything with a computable answer: write the expect test with the hand-computed number, watch it fail, then make the diff vanish. Never promote an empty expect block blindly. Read every promoted diff — a surprising one is a real signal.
- **Layout** lib/<name>/src and lib/<name>/test, library execlab_<name>, with a top-level re-export module.
- **Formatting** dune fmt --auto-promote, janestreet profile, 77-column margin.
- **CI** Builds and tests the engine and CLI on the stock public toolchain. The browser client is excluded because it needs Jane Street preview packages and the OCaml toolchain, which the public opam repository does not carry.

- **Context** context.md is the project bible — the plan, every design decision with its rationale, the current state and the roadmap. Read it first and keep it current.

14. Where the project stands

Everything through the original roadmap is built and merged: all five algorithms, both fill engines, the full cost decomposition, the eight-screen client, accounts and a server-verified leaderboard, CSV export, the demo library and the invariant sweep. The sweep currently reports 5,940 runs with zero violations.

Known open items: multi-symbol alpha files (a run currently binds to one symbol and one trading day all the way down — supporting several would mean the driver stepping multiple sessions in lockstep on one clock, per-symbol engines and statistics, and a portfolio spanning symbols); a js_of_ocaml release build to shrink the roughly 70MB development bundle; prior-days-only VWAP forecasts once users can pick dates freely; and retiring the older standalone client under app/ui/client now that its screens are integrated.